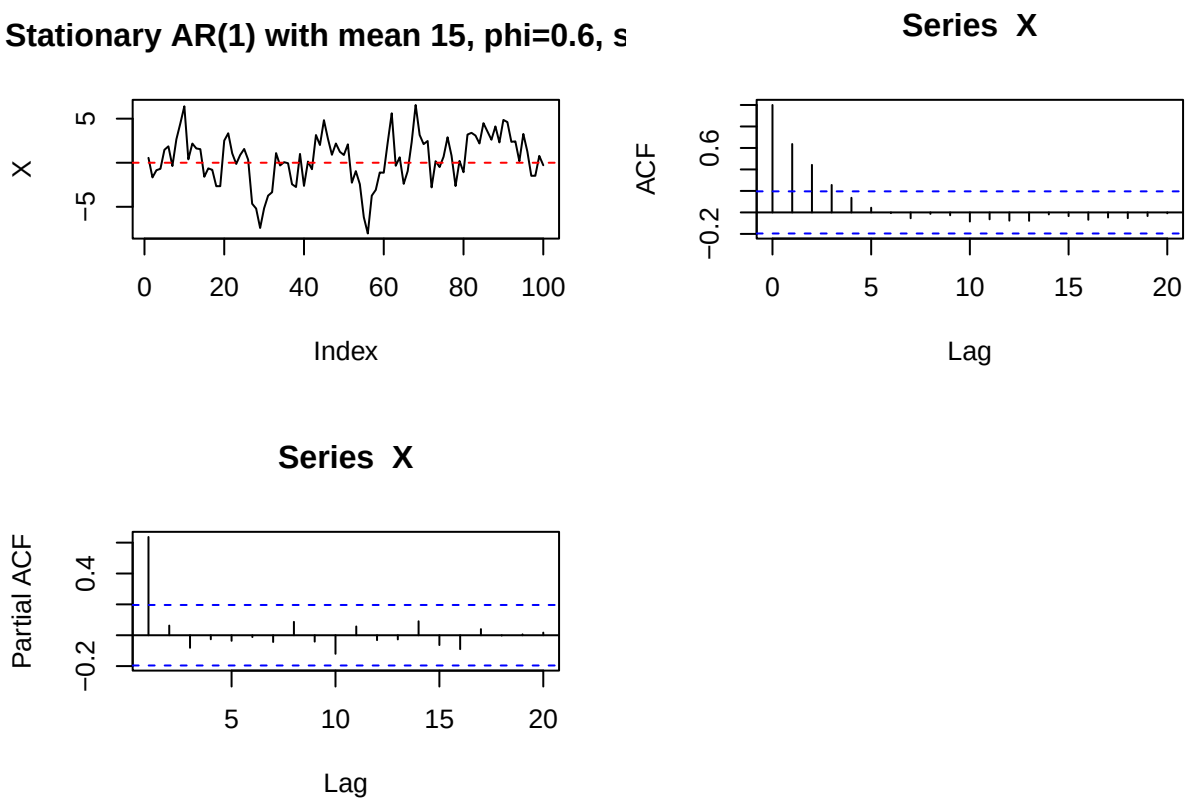
Kalman Filter Training

Basic Kalman Filtering

Time series Setup

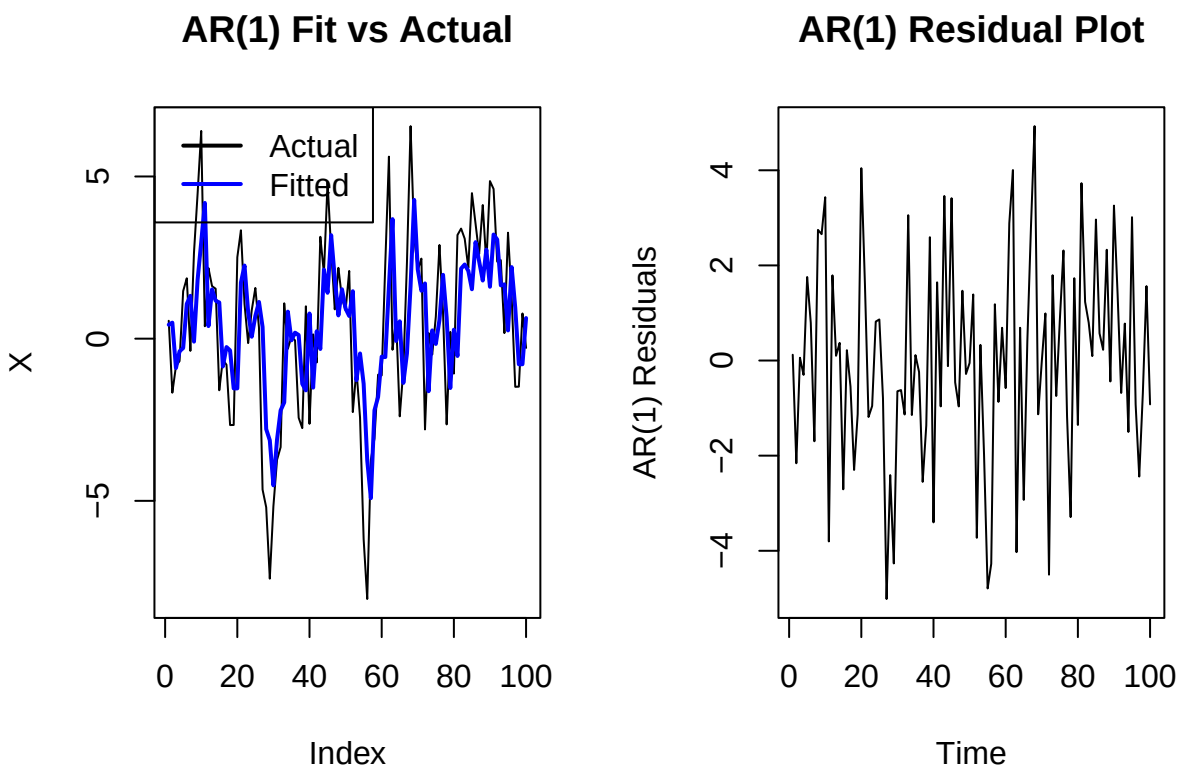
First, lets create some time series Data. Its verifiable that it is a short time series by the use of the ACF which shows a quick decay.

$$x_t = \phi x_{t-1} + w_t$$
$$\phi = 0.6, w_t \sim N(0, 5)$$



AR(1) Walkthrough

Now we take a quick peek at the AR(1) Example of the data



Kalman Filtering

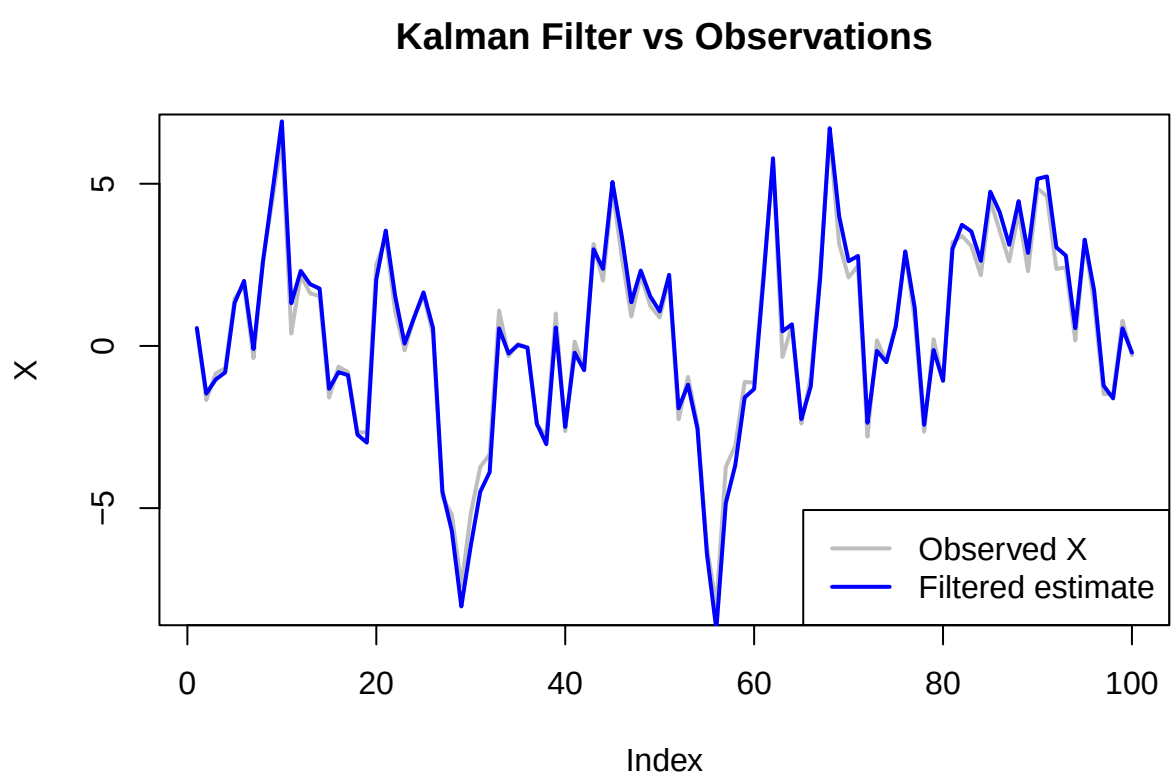
Now onto the Kalman filter itself! This is best explained by the parameters K and P. If K and P are big, you rely heavily on the data. As it goes on, the P and K will decrease and smooth it out. Here's some math

Prediction step. let t-1 be the most recent observed value we have, and we want to predict the value of x at t

$$\hat{X}_{t|t-1} = \phi \hat{x}_{t-1|t-1}$$
$$P_{t|t-1} = \phi^2 P_{t-1|t-1} + Q$$

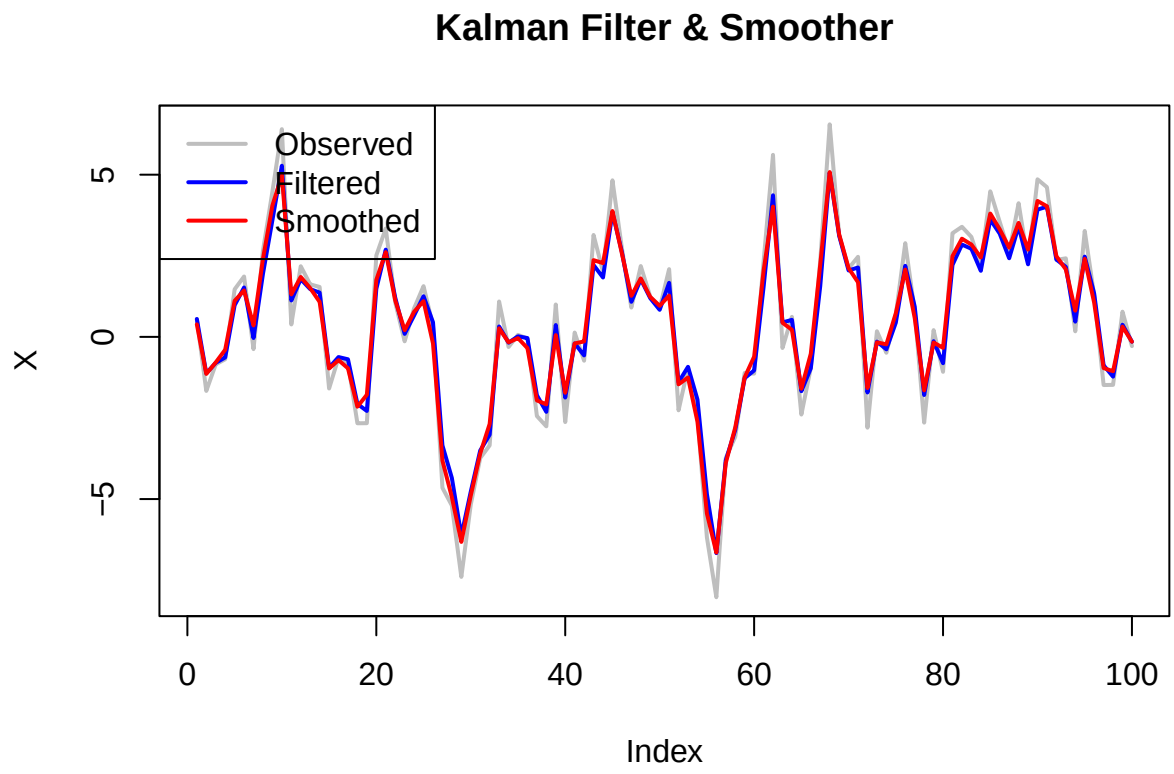
Ok, so you did the prediction, some time passed and you acquired the true value. What to do next? We update based on the Kalman Gain, which is a parameter that basically asks, "How good did we do?" and allows you to change your weight of reliance on prior information

$$K_t = \frac{P_{t|t-1}}{(P_{t|t-1} + R)}$$
$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(y_t - \hat{x}_{t|t-1})$$
$$P_{t|t} = (1 - K)P_{t|t-1}$$



Kalman Smoothing

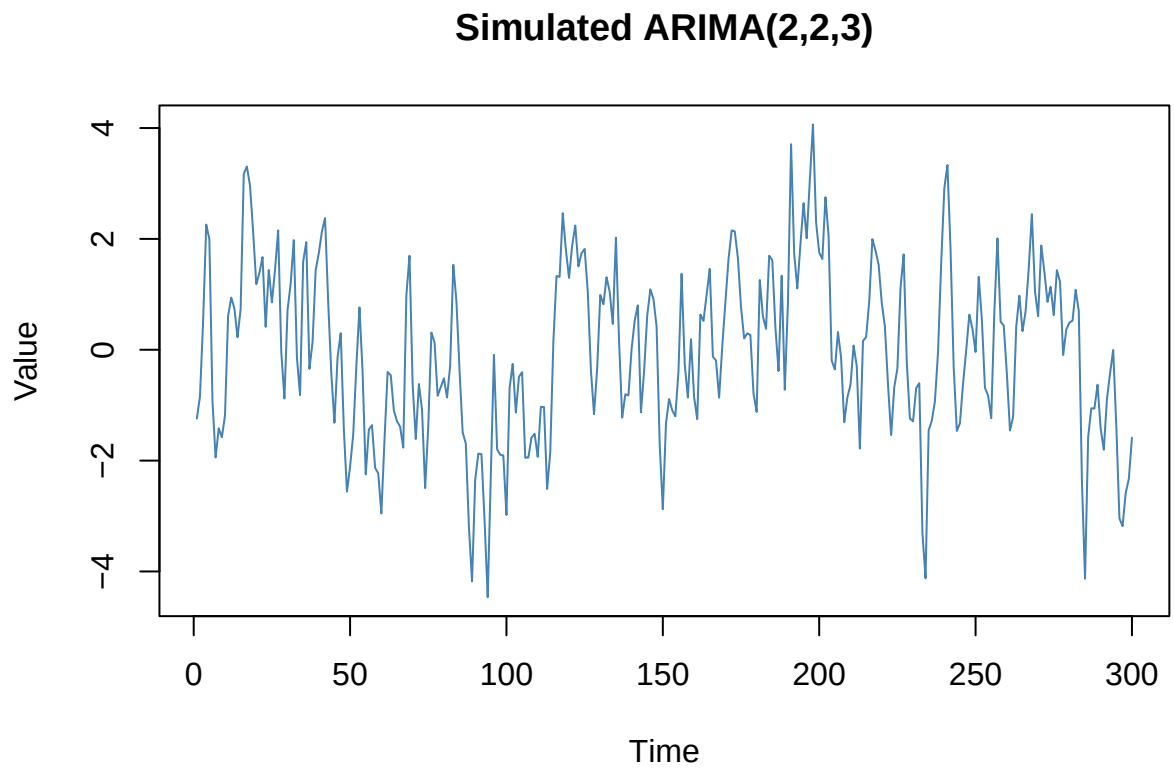
```r
phi <- 0.6
Q <- 5
R <- 2
n <- length(X)

# --- Forward pass (filtering) ---
xhat <- numeric(n)
P <- numeric(n)
x_pred <- numeric(n)
P_pred <- numeric(n)

xhat[1] <- X[1]
P[1] <- 1

for (t in 2:n) {
  # Predict
  x_pred[t] <- phi * xhat[t - 1]
  P_pred[t] <- phi^2 * P[t - 1] + Q

  # Update
  K <- P_pred[t] / (P_pred[t] + R)
  xhat[t] <- x_pred[t] + K * (X[t] - x_pred[t])
  P[t] <- (1 - K) * P_pred[t]
}

# --- Backward pass (smoothing) ---
x_smooth <- xhat
P_smooth <- P

for (t in (n - 1):1) {
  A <- P[t] * phi / P_pred[t + 1]
  x_smooth[t] <- xhat[t] + A * (x_smooth[t + 1] - x_pred[t + 1])
  P_smooth[t] <- P[t] + A^2 * (P_smooth[t + 1] - P_pred[t + 1])
}

# --- Plot results ---
plot(X, type = "l", col = "gray", lwd = 2, main = "Kalman Filter & Smoother")
lines(xhat, col = "blue", lwd = 2)
lines(x_smooth, col = "red", lwd = 2)
legend("topleft", legend = c("Observed", "Filtered", "Smoothed"),
       col = c("gray", "blue", "red"), lty = 1, lwd = 2)
```

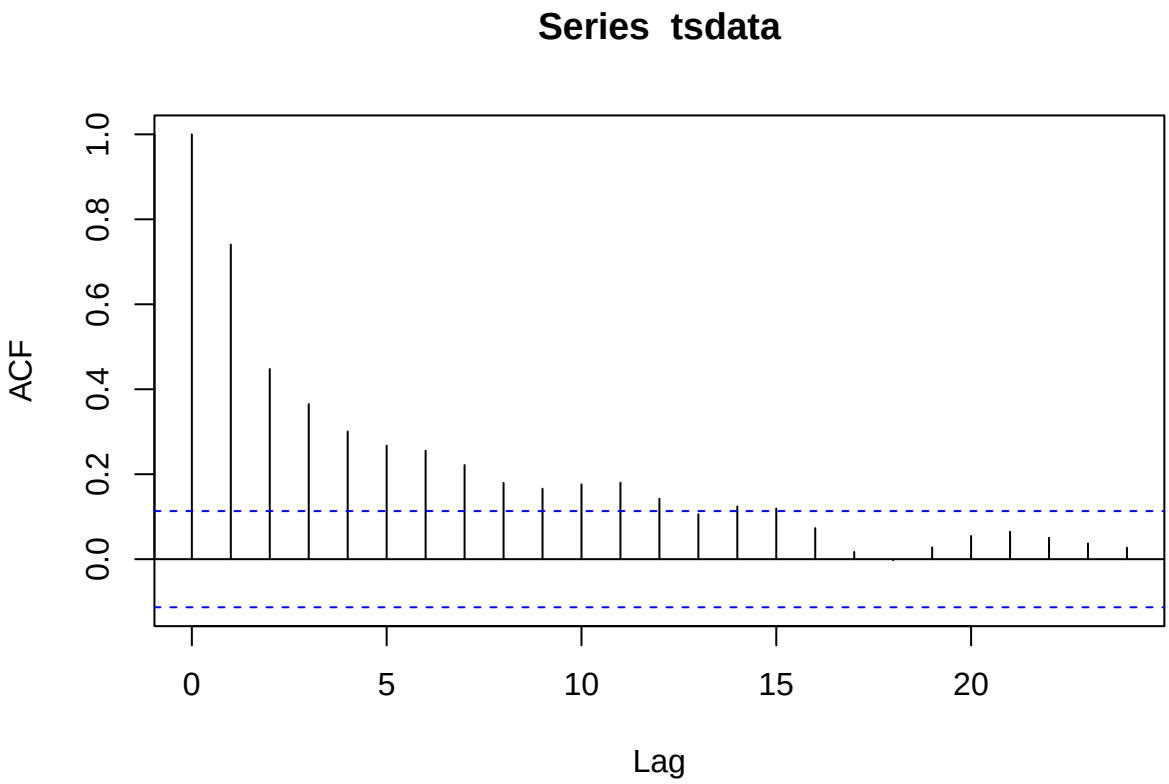


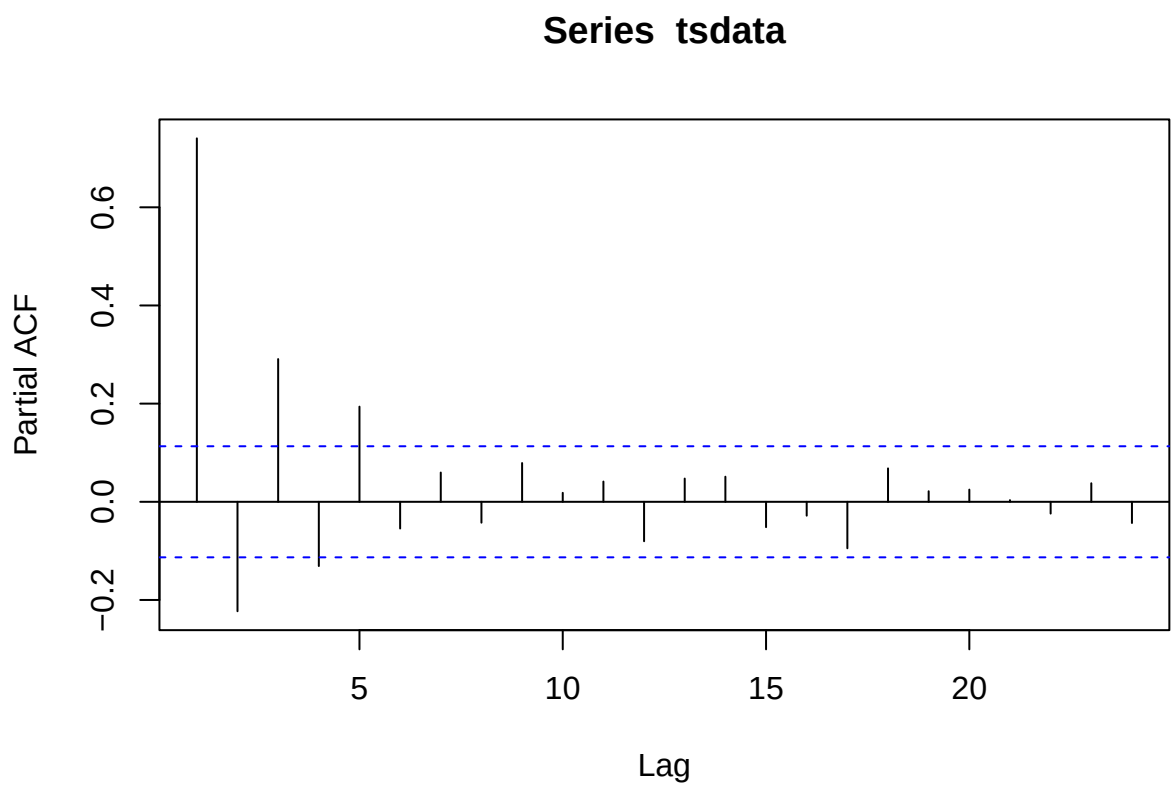
ARMA(1,2) Example

Data Creation

This time we forego the independent creation of data and use a conventional one. ARIMA.sim(2,2,3) to create a more complex situation


Simulated ARIMA(2,2,3)
Value
Time
4
2
0
−2
−4
0
50
100
150
200
250
300


Series tsdata
ACF
1.0
0.8
0.6
0.4
0.2
0.0
0
5
10
15
20
Lag



Fitting using the model

Spot on! Good work! Our ARMA(1,2) model perfectly fits out simulated ARMA(1,2) data!

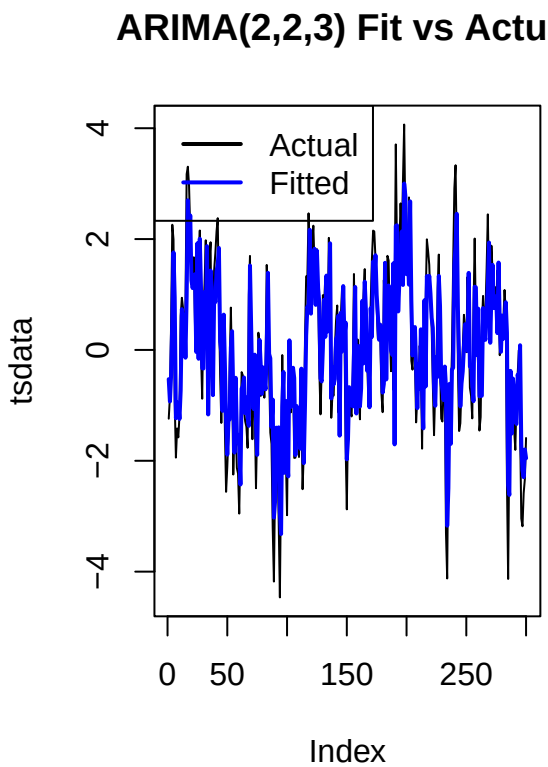
```
ARMAModel <- arima(tsdata, order = c(1,0,2))
ARMAModel
```

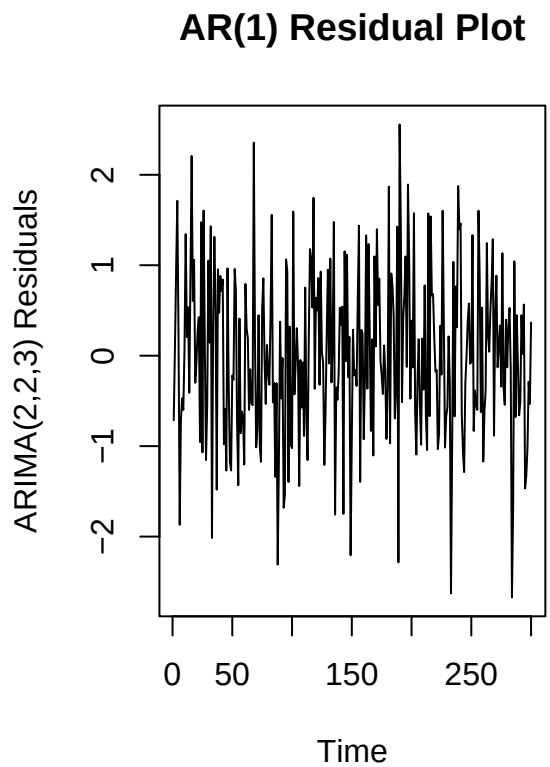
```
##
## Call:
## arima(x = tsdata, order = c(1, 0, 2))
##
## Coefficients:
##          ar1     ma1      ma2  intercept
##       0.8476  0.1897  -0.4549    -0.0651
## s.e.  0.0692  0.1058   0.0985     0.2536
##
## sigma^2 estimated as 0.8522:  log likelihood = -402.44,  aic = 814.88
```

```
par(mfrow=c(1,2))
MSE = mean(ARMAModel$residuals^2)
print(paste("MSE of model is:", MSE))
```

```
## [1] "MSE of model is: 0.852211847476978"
```

```r
fitted_vals = fitted(ARMAModel)
plot(tsdata, type = "l", col = "black", main = "ARIMA(2,2,3) Fit vs Actual")
lines(fitted_vals, col = "blue", lwd = 2)
legend("topleft", legend = c("Actual", "Fitted"),
       col = c("black", "blue"), lty = 1, lwd = 2)
plot(ARMAModel$residuals, ylab="ARIMA(2,2,3) Residuals", main="AR(1) Residual Plot")
```

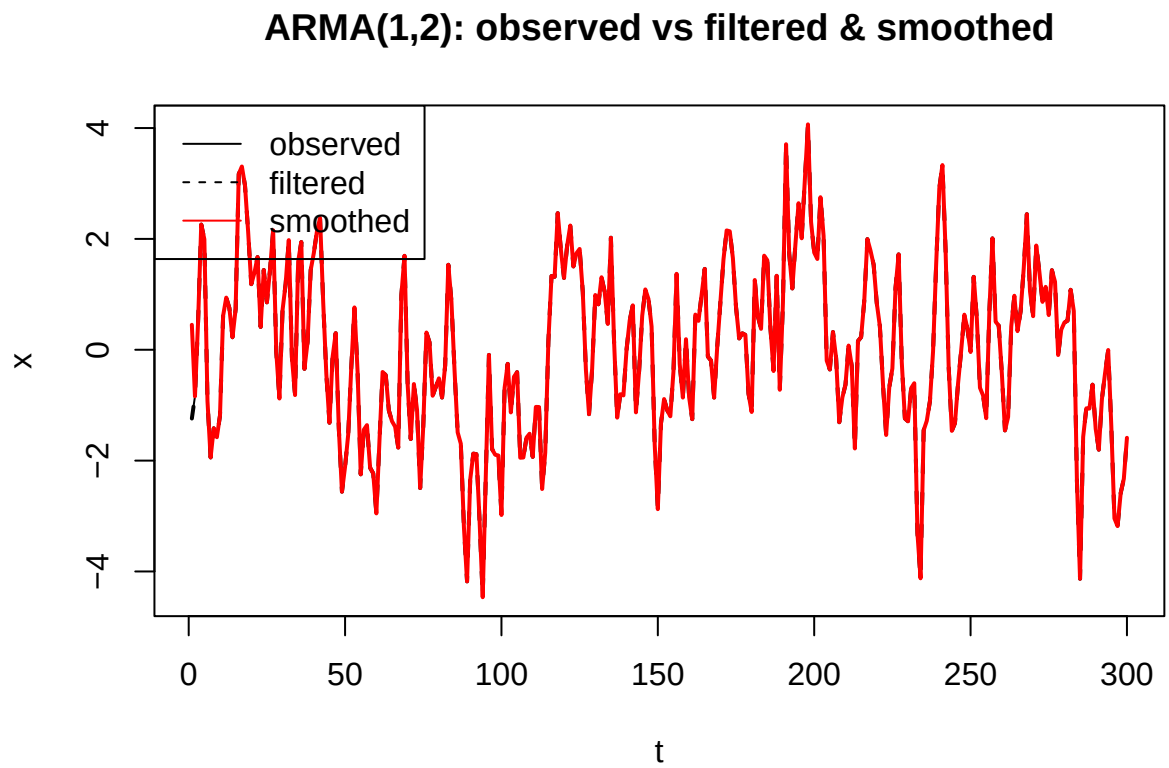




And for the kalman filter

```r
# ---------- params ----------
n <- 300
phi    <- 0.7
theta1 <- 0.4
theta2 <- -0.25
sigma2 <- 1     # innovation variance
Robs   <- 0.0001  # observation noise variance (tiny; set 0 if you want exact obs)


# ---------- state-space matrices ----------
F <- matrix(c(phi, theta1, theta2,
              0,   0,      0,
              0,   1,      0), nrow=3, byrow=TRUE)
B <- matrix(c(1, 1, 0), nrow=3)
```

```r
H <- matrix(c(1,0,0), nrow=1)
Q <- sigma2 * (B %*% t(B))
R <- Robs

# ---------- allocate storage ----------
Tn <- length(tsdata)
m <- nrow(F)
x_pred <- matrix(0, m, Tn)   # x_{t|t-1}
P_pred <- array(0, c(m,m,Tn))
x_filt <- matrix(0, m, Tn)   # x_{t|t}
P_filt <- array(0, c(m,m,Tn))

# ---------- init ----------
x_filt[,1] <- c(tsdata[1], 0, 0)   # initial guess (observed first value)
P_filt[,,1] <- diag(100, m)        # large uncertainty to start

# ---------- forward Kalman filter ----------
for (t in 2:Tn) {
  # predict
  x_pred[,t] <- F %*% x_filt[,t-1]
  P_pred[,,t] <- F %*% P_filt[,,t-1] %*% t(F) + Q

  # update using y_t
  S <- H %*% P_pred[,,t] %*% t(H) + R
  K <- P_pred[,,t] %*% t(H) %*% solve(S)   # Kalman gain (m x 1)
  innov <- tsdata[t] - (H %*% x_pred[,t])
  x_filt[,t] <- x_pred[,t] + K %*% innov
  P_filt[,,t] <- (diag(m) - K %*% H) %*% P_pred[,,t]
}

# ---------- Rauch-Tung-Striebel smoother (backwards) ----------
x_smooth <- x_filt
P_smooth <- P_filt
for (t in (Tn-1):1) {
  C <- P_filt[,,t] %*% t(F) %*% solve(P_pred[,,t+1])
  x_smooth[,t] <- x_filt[,t] + C %*% (x_smooth[,t+1] - x_pred[,t+1])
  P_smooth[,,t] <- P_filt[,,t] + C %*% (P_smooth[,,t+1] - P_pred[,,t+1]) %*% t(C)
}

# ---------- extract */
x_filtered <- as.numeric(x_filt[1,])
x_smoothed <- as.numeric(x_smooth[1,])

# ---------- plots ----------
plot(tsdata, type="l", main="ARMA(1,2): observed vs filtered & smoothed", ylab="x", xlab="t")
lines(x_filtered, lty=2, lwd=2)
lines(x_smoothed, col="red", lwd=2)
legend("topleft", legend=c("observed","filtered","smoothed"), lty=c(1,2,1), col=c("black","black","red")
```




```r
MSE = mean((tsdata-x_filtered)^2)
print(paste("MSE of model is:", MSE))
```

```
## [1] "MSE of model is: 8.55428922562358e-09"
```

ARTFIMA Example